

UNIVERSIDADE DA BEIRA INTERIOR
Covilhã | Portugal

Ecossistema Web, Móvel e na Nuvem - Trabalho Final

Afonso Esteves - 55047
Duarte Azenhas - 55281
Felipe Dourado - 53394

26/05/2025

Conteúdo

Resumo **4**

1 Introdução **5**
1.1 Âmbito, enquadramento e motivação 5

2 Contextualização do Projeto **6**
2.1 Identificação dos Desenvolvedores 6
2.1.1 Desenvolvedores Envolvidos 6
2.2 Contextualização Académica 6

3 Especificações Técnicas **7**
3.1 Ambiente de Desenvolvimento 7
3.2 Dependências e Bibliotecas 7

4 Usabilidade **8**
4.1 Requisitos Prévios 8
4.2 Como executar o programa 8
4.3 Uso do Programa 9

5 Estrutura do Programa **9**
5.1 Componentes do Sistema 9

6 Funcionalidades Implementadas **10**

7 Implementação das Funcionalidades **11**
7.1 Funcionalidade: Rota Principal 11
7.2 Funcionalidade: Tabela de Código de *Gray* 11
7.2.1 Rota: `/gray` 11
7.2.2 Função: `codigo_binario(n)` 12
7.2.3 Função: `gerar_tabela_gray(n)` 12
7.2.4 Função: `codigo_gray(n)` 13
7.3 Funcionalidade: Conversão entre Bases Numéricas 14
7.4 Funcionalidade: Operações Aritméticas 15
7.5 Funcionalidade: Conversão de Bases Numéricas 15
7.5.1 Rota: `/conversao` 16
7.5.2 Validações e Tratamento de Erros 18
7.6 Funcionalidade: Operações Aritméticas 18
7.6.1 Rota: `/operacoes` 18
7.6.2 Validações e Tratamento de Erros 20
7.6.3 Resultado da Operação 21

8 Considerações de Implementação **22**
8.1 Princípios de Desenvolvimento 22
8.2 Desafios Encontrados 22
8.3 Soluções Arquiteturais 23

9 Conclusão **24**

10 Referências Bibliográficas **25**

11 Anexos **26**
11.1 Estrutura de Ficheiros do Projeto 26
11.2 Interfaces da Aplicação 26

Resumo

Este relatório descreve o desenvolvimento de uma aplicação web responsiva desenvolvida com Flask (Python) como *back-end* e React (JavaScript) como *front-end*. O sistema implementa funcionalidades fundamentais relacionadas com sistemas numéricos, incluindo a geração de tabelas de Código de *Gray*, conversão entre diferentes bases numéricas (binário, decimal e hexadecimal) e execução de operações aritméticas básicas em representações binárias e hexadecimais. A aplicação utiliza uma arquitetura moderna de separação entre *front-end* e *back-end*, proporcionando uma experiência de utilizador intuitiva e responsiva através de uma interface web moderna.

Palavras-chave e *Keywords*

flask; react; tabela de gray; conversão de bases; sistemas numéricos; aritmética binária; aritmética hexadecimal; aplicação web; *front-end* responsivo.

1 Introdução

1.1 Âmbito, enquadramento e motivação

Este documento consiste num relatório técnico que descreve o desenvolvimento, execução e validação de uma aplicação web implementada com Flask (Python) para o *back-end* e React (JavaScript) para o *front-end*. A aplicação tem como objetivo principal disponibilizar através de uma interface web intuitiva várias funcionalidades relacionadas com sistemas numéricos, incluindo geração de tabelas de *Gray*, conversão entre bases numéricas e operações aritméticas.

O contexto deste projeto insere-se no ensino de ecossistemas web, destacando a aplicação prática de conceitos fundamentais de desenvolvimento web full-stack. A convergência entre teoria e prática da disciplina permitiu a criação e desenvolvimento de competências em arquitetura de aplicações web, separação de responsabilidades entre *front-end* e *back-end*, e implementação de interfaces utilizador modernas e responsivas.

O projeto situa-se na área de Desenvolvimento Web Full-Stack, mais especificamente nas sub-áreas de Desenvolvimento *Back-End* com Flask, *Front-End* Moderno com React, e Processamento de Dados Numéricos. A arquitetura adotada organiza o código de forma modular e escalável, enquanto o processamento de dados permite a manipulação, transformação e apresentação de informações numéricas de forma eficiente.

A importância técnica de abordar o problema reside na necessidade de desenvolver soluções web modernas que validem entradas de utilizadores, processem dados de forma eficiente no servidor, e apresentem resultados através de interfaces responsivas e interativas. Além disso, permite um amplo conhecimento em técnicas de desenvolvimento web full-stack e integração entre diferentes tecnologias.

2 Contextualização do Projeto

2.1 Identificação dos Desenvolvedores

Os autores do presente projeto são estudantes do curso de Informática Web, Móvel e Nuvem da Universidade da Beira Interior (UBI), matriculados na disciplina de Ecossistema Web, Móvel e na Nuvem. Cada membro contribuiu significativamente para o desenvolvimento do sistema, aplicando conhecimentos teóricos e práticos adquiridos durante o curso.

2.1.1 Desenvolvedores Envolvidos

- **Afonso Esteves (55047)**
- **Duarte Azenhas (55281)**
- **Felipe Dourado (53394)**

2.2 Contextualização Académica

O projeto foi desenvolvido no contexto da disciplina de Ecossistema Web, Móvel e na Nuvem, com o objetivo de aplicar e consolidar conhecimentos em:

- Desenvolvimento web full-stack
- Arquitetura de aplicações web modernas
- Integração entre *back-end* (Flask) e *front-end* (React)
- Desenvolvimento de interfaces utilizador responsivas
- Processamento de dados no servidor
- Boas práticas de desenvolvimento web

3 Especificações Técnicas

3.1 Ambiente de Desenvolvimento

- **Linguagem *back-end*:** Python 3.9.6
- **Framework *back-end*:** Flask
- **Linguagem *front-end*:** JavaScript (ES6+)
- **Biblioteca *front-end*:** React 18
- **Transpilador:** Babel (para JSX)
- **Framework CSS:** Tailwind CSS
- **Template Engine:** Jinja2 (integrado com Flask)
- **Ambientes de Desenvolvimento:** Visual Studio Code, GitHub
- **Sistemas Operacionais:** Windows, macOS e Linux

3.2 Dependências e Bibliotecas

***Back-End* (Flask):**

- Flask (framework web)
- Jinja2 (template engine)

***Front-End* (React):**

- React 18 (biblioteca de interface utilizador)
- ReactDOM (renderização DOM)

Styling:

- Tailwind CSS (framework CSS utilitário)

4 Usabilidade

4.1 Requisitos Prévios

- **Python:** É necessário ter o Python 3.9.6 instalado no sistema operacional.
- **Flask:** Instalação via pip
- **Navegador Web:** Qualquer navegador moderno com suporte a ES6+

4.2 Como executar o programa

- **Preparação do Ambiente:**
  1. Navegue até à diretoria do projeto.
  2. Instale as dependências:
- **Execução:**
  1. Crie um ambiente virtual Python desta forma:

     ```
     python -m venv .venv
     ```

  2. Instale as dependências:

     ```
     pip3 install flask
     ```

  3. Defina as variáveis de ambientes para o Flask:

     ```
     export FLASK_APP=app
     ```

  4. Execute a aplicação:

     ```
     flask run
     ```

- **Acesso:**
  - Abra o navegador e aceda a `http://localhost:5000`

4.3 Uso do Programa

- Ao aceder à aplicação, será apresentada uma página inicial com três opções principais;
- Selecione uma das funcionalidades disponíveis: Tabela de *Gray*, Conversões ou Operações;
- Cada funcionalidade possui uma interface intuitiva com formulários e apresentação de resultados em tempo real;
- Para voltar ao menu principal, utilize o botão de navegação presente em cada página.

5 Estrutura do Programa

O sistema foi arquitetado seguindo princípios de separação de responsabilidades e desenvolvimento web moderno, dividindo-se em componentes distintos para *back-end* e *front-end*:

5.1 Componentes do Sistema

***back-end* (Flask):**

- `app.py`: Servidor Flask principal com rotas e lógica de negócio
- Rotas implementadas: `/`, `/gray`, `/conversao`, `/operacoes`

***front-end* (React):**

- `index.html`: Página principal com menu de navegação
- `gray.html`: Interface para geração de tabelas de *Gray*
- `conversao.html`: Interface para conversão entre bases numéricas
- `operacoes.html`: Interface para operações aritméticas

Assets:

- Imagens da UBI (logótipo e fundo)
- Estilos CSS integrados com Tailwind

6 Funcionalidades Implementadas

As funcionalidades implementadas na aplicação web incluem:

- Criação de tabelas em código de *Gray* de n bits
  - Interface web para inserção do número de bits (1-10)
  - Geração automática da tabela com valores decimais, binários e *Gray*
  - Apresentação em formato de tabela responsiva
- Conversão entre sistemas numéricos:
  - Conversão bidirecional entre:
    - Binário $\longleftrightarrow$ Decimal
    - Decimal $\longleftrightarrow$ Hexadecimal
    - Binário $\longleftrightarrow$ Hexadecimal
  - Interface intuitiva com dropdowns para seleção de bases
  - Validação de entrada em tempo real
- Resolução de operações aritméticas:
  - Adição e subtração em sistema binário
  - Adição e subtração em sistema hexadecimal
  - Interface com visualização da operação em tempo real
  - Apresentação do resultado na base selecionada

7 Implementação das Funcionalidades

7.1 Funcionalidade: Rota Principal

Esta funcionalidade representa a rota inicial da aplicação web desenvolvida com **Flask** no *back-end*. Quando o utilizador acessa a raiz do domínio, a função associada à rota `"/"` é invocada e renderiza a página principal da interface gráfica.

O conteúdo exibido é gerado a partir do ficheiro `index.html`, localizado na pasta `templates/`. Este ficheiro contém a estrutura inicial da aplicação web, sendo responsável por carregar o ***front-end*** desenvolvido com **React**, estilizado com **Tailwind CSS** e compilado com **Babel**. Esta abordagem permite a criação de uma interface dinâmica e responsiva, promovendo uma boa experiência de utilizador.

```
@app.route("/")
def index():
        return render_template("index.html")
```

7.2 Funcionalidade: Tabela de Código de *Gray*

Esta funcionalidade permite ao utilizador gerar uma tabela com representações decimais, binárias e em Código de *Gray* para um número específico de bits. A funcionalidade está integrada à interface web da aplicação, construída com **Flask** no *back-end* e **React** com **Tailwind CSS** no *front-end*.

7.2.1 Rota: `/gray`

A rota `/gray` é responsável por lidar com requisições `GET` e `POST`. Quando o método for `POST`, o sistema lê o valor inserido pelo utilizador (número de bits), converte-o para inteiro e chama a função `gerar_tabela_gray`. Em caso de erro na conversão, a tabela é retornada vazia. O resultado final é renderizado na página `gray.html`.

```
@app.route("/gray", methods=["GET", "POST"])
def gray():
    tabela = []
```

```
    n = None
    if request.method == "POST":
        try:
            n = int(request.form.get("bits", 0))
            tabela = gerar_tabela_gray(n)
        except ValueError:
            tabela = []
    return render_template("gray.html", tabela=tabela, n=n)
```

7.2.2 Função: `codigo_binario(n)`

Esta função gera todos os números binários com n bits, utilizando a função `format` do Python. O resultado é uma lista com todos os números de 0 até $2^n - 1$ no formato binário, preenchido com zeros à esquerda.

```
def codigo_binario(n):
    return [format(i, f"0{n}b") for i in range(2**n)]
```

7.2.3 Função: `gerar_tabela_gray(n)`

Esta função monta a tabela com três colunas: o número decimal, a sua representação binária e o correspondente em Código de *Gray*. Para isso, utiliza as funções auxiliares `codigo_binario` e `codigo_gray`.

```
def gerar_tabela_gray(n):
    if n <= 0:
        return []

    codigos = codigo_gray(n)
    binarios = codigo_binario(n)

    tabela = []
    for i in range(len(codigos)):
        tabela.append((i, binarios[i], codigos[i]))

    return tabela
```

7.2.4 Função: `codigo_gray(n)`

Esta função gera a sequência de Código de *Gray* para n bits, utilizando um algoritmo recursivo. A lógica baseia-se na construção do código de n bits a partir do código de $n-1$ bits, onde a primeira metade da nova sequência recebe o prefixo `"0"` e a segunda metade (espelhada) recebe o prefixo `"1"`.

```
def codigo_gray(n):
    if n <= 0:
        return []

    if n == 1:
        return ["0", "1"]

    gray_menor = codigo_gray(n-1)
    resultado = []

    for codigo in gray_menor:
        resultado.append("0" + codigo)

    for codigo in reversed(gray_menor):
        resultado.append("1" + codigo)

    return resultado
```

7.3 Funcionalidade: Conversão entre Bases Numéricas

O sistema de conversões permite transformar números entre diferentes bases numéricas através de uma interface web intuitiva. As conversões são realizadas no servidor Flask, garantindo precisão e validação adequada dos dados.

```python
def codigo_binario(n):
    if n <= 0:
        return []

    if n == 1:
        return ["0", "1"]

    binario_menor = codigo_binario(n-1)
    resultado = []

    for codigo in binario_menor:
        resultado.append("0" + codigo)

    for codigo in binario_menor:
        resultado.append("1" + codigo)

    return resultado
```

Implementação:

- Rota Flask `/conversao` com lógica de conversão centralizada
- Suporte para todas as combinações entre binário, decimal e hexadecimal
- Validação de entrada específica para cada base numérica
- Interface React com dropdowns dinâmicos e feedback em tempo real

7.4 Funcionalidade: Operações Aritméticas

Esta funcionalidade oferece a capacidade de realizar operações aritméticas básicas (adição e subtração) em sistemas binário e hexadecimal, com apresentação dos resultados na base selecionada.

```
def binario_decimal(binary):
    decimal = 0
    troca = 0

    for digito in reversed(binary):
        if digito == '1':
            decimal += 2 ** troca
        troca += 1

    return decimal
```

Implementação:

- Rota Flask `/operacoes` com processamento de operações no servidor
- Conversão para decimal como passo intermediário para garantir precisão
- Interface React com visualização da operação matemática
- Tratamento de números negativos e validação de entrada

7.5 Funcionalidade: Conversão de Bases Numéricas

Esta funcionalidade permite ao utilizador realizar conversões entre diferentes bases numéricas através da interface web. O *back-end* da aplicação é desenvolvido com **Flask**, enquanto o *front-end* é implementado com **React** e estilizado com **Tailwind CSS**, sendo transpilado com **Babel**.

A interface oferece ao utilizador a possibilidade de escolher a base de origem e a base de destino entre as seguintes opções:

- Binário
- Decimal
- Hexadecimal

O utilizador também pode inserir o valor a ser convertido. O sistema realiza a validação dos dados de entrada e efetua a conversão apropriada, exibindo o resultado na própria página.

7.5.1 Rota: `/conversao`

A rota `/conversao` trata requisições `GET` e `POST`. Quando o método é `POST`, os dados fornecidos pelo utilizador são processados e convertidos entre as bases especificadas. A resposta é renderizada na página `conversao.html` com o resultado.

```
@app.route("/conversao", methods=["GET", "POST"])
def conversao():
    resultado = ""
    de = para = valor = ""

    if request.method == "POST":
        de = request.form.get("de")
        para = request.form.get("para")
        valor = request.form.get("valor", "").strip()

        if not valor:
            resultado = "Por favor, insira um valor."
            return render_template("conversao.html",
                resultado=resultado, de=de, para=para, valor=valor)

        try:
            if de == "binario" and para == "decimal":
                if not all(c in '01' for c in valor):
                    raise ValueError("Valor bin rio inv lido")
                resultado = int(valor, 2)

```

```
            elif de == "decimal" and para == "binario":
                resultado = bin(int(valor))[2:]

            elif de == "decimal" and para == "hexadecimal":
                resultado = hex(int(valor))[2:].upper()

            elif de == "hexadecimal" and para == "decimal":
                try:
                    resultado = int(valor, 16)
                except ValueError:
                    raise ValueError("Valor hexadecimal inv lido")

            elif de == "binario" and para == "hexadecimal":
                if not all(c in '01' for c in valor):
                    raise ValueError("Valor bin rio inv lido")
                decimal = int(valor, 2)
                resultado = hex(decimal)[2:].upper()

            elif de == "hexadecimal" and para == "binario":
                decimal = int(valor, 16)
                resultado = bin(decimal)[2:]

            elif de == para:
                resultado = valor
            else:
                resultado = "Convers o  n  o  suportada."

        except ValueError as e:
            resultado = f"Erro: {str(e)}"
        except Exception as e:
            resultado = "Erro na convers o. Verifique o valor
                inserido."

    return render_template("conversao.html", resultado=resultado,
        de=de, para=para, valor=valor)
```

7.5.2 Validações e Tratamento de Erros

Durante o processo de conversão, são realizadas diversas validações:

- Verificação se o valor foi inserido.
- Verificação de validade para números binários (somente dígitos `0` e `1`).
- Validação de números hexadecimais usando `try/except`.
- Tratamento de casos onde a base de origem e destino são iguais.
- Exibição de mensagens de erro apropriadas ao utilizador em caso de entrada inválida.

Este nível de verificação garante maior robustez ao sistema, evitando falhas de conversão e melhorando a experiência do utilizador.

7.6 Funcionalidade: Operações Aritméticas

Esta funcionalidade permite que o utilizador realize operações matemáticas simples (`+` e `-`) entre dois números fornecidos, que podem estar representados em diferentes sistemas numéricos: **Binário** ou **Hexadecimal**.

O sistema converte os números para a base decimal, realiza a operação aritmética e depois converte o resultado de volta para o sistema selecionado, mantendo a interface web responsiva e amigável construída com **React**, **Tailwind CSS** e **Flask** como *back-end*.

7.6.1 Rota: `/operacoes`

A rota `/operacoes` aceita requisições `GET` e `POST`, e renderiza a página `operacoes.html` com os dados processados ou mensagens de erro, caso existam.

```
1 @app.route("/operacoes", methods=["GET", "POST"])
2 def operacoes():
3     resultado = ""
4     sistema = operacao = valor1 = valor2 = ""
```

```
5
6       if request.method == "POST":
7           sistema = request.form.get("sistema")
8           operacao = request.form.get("operacao")
9           valor1 = request.form.get("valor1", "").strip().upper()
10          valor2 = request.form.get("valor2", "").strip().upper()
11
12          try:
13              if not sistema:
14                  raise ValueError("Por favor, selecione um sistema
                        num rico")
15
16              if sistema == "Binario":
17                  if not all(c in '01' for c in valor1) or not all(c
                        in '01' for c in valor2):
18                      raise ValueError("Valores bin rios
                            inv lidos. Use apenas 0 e 1")
19                  decimal1 = int(valor1, 2)
20                  decimal2 = int(valor2, 2)
21
22              elif sistema == "Hexadecimal":
23                  try:
24                      decimal1 = int(valor1, 16)
25                      decimal2 = int(valor2, 16)
26                  except ValueError:
27                      raise ValueError("Valores hexadecimais
                            inv lidos")
28              else:
29                  raise ValueError("Sistema num rico  n  o
                        suportado")
30
31              if operacao == "+":
32                  resultado_decimal = decimal1 + decimal2
33              elif operacao == "-":
34                  resultado_decimal = decimal1 - decimal2
35              else:
36                  raise ValueError("Opera  o  n  o  suportada")
37
38              if sistema == "Binario":
39                  resultado = {
40                      'valor': bin(abs(resultado_decimal))[2:],
41                      'sistema': 'Bin rio',
42                      'negativo': resultado_decimal < 0
```

```
43                 }
44             elif sistema == "Hexadecimal":
45                 resultado = {
46                     'valor':
                           hex(abs(resultado_decimal))[2:].upper(),
47                     'sistema': 'Hexadecimal',
48                     'negativo': resultado_decimal < 0
49                 }
50
51         except ValueError as e:
52             resultado = {'erro': str(e)}
53         except Exception:
54             resultado = {'erro': 'Erro na opera  o. Verifique os
                   valores inseridos.'}
55
56     return render_template("operacoes.html", resultado=resultado,
           sistema=sistema, operacao=operacao, valor1=valor1,
           valor2=valor2)
```

7.6.2 Validações e Tratamento de Erros

Diversas validações são realizadas antes de efetuar a operação:

- Verificação se o sistema numérico foi selecionado.
- Verificação se os valores são válidos conforme o sistema (binário ou hexadecimal).
- Conversão de valores para decimal para garantir a correta operação.
- Deteção de operação suportada (`+` ou `-`).
- Tratamento de valores negativos, com conversão apropriada.
- Exibição de mensagens específicas para entradas inválidas.

7.6.3 Resultado da Operação

O resultado é devolvido como um dicionário contendo:

- `valor`: valor absoluto da operação convertido para o sistema selecionado.
- `sistema`: nome do sistema utilizado.
- `negativo`: booleano indicando se o resultado é negativo.

Esse formato permite à interface apresentar o valor resultante corretamente, inclusive indicando o sinal quando necessário.

8 Considerações de Implementação

8.1 Princípios de Desenvolvimento

- **Separação de Responsabilidades:** *back-end* Flask para lógica de negócio, *front-end* React para interface utilizador
- **Responsividade:** Interface adaptável a diferentes dispositivos
- **Validação de Dados:** Validação tanto no *front-end* quanto no *back-end*
- **Experiência do Utilizador:** Interface intuitiva com feedback visual
- **Modularidade:** Código organizado em rotas e componentes reutilizáveis

8.2 Desafios Encontrados

Durante o desenvolvimento do projeto, diversos desafios técnicos e arquiteturais surgiram, exigindo soluções criativas e bem estruturadas. Abaixo estão os principais obstáculos enfrentados:

- **Integração entre Flask e React:** Conciliar a lógica de renderização do *template engine* Jinja2 (utilizado pelo Flask) com os componentes JSX do React foi um dos maiores desafios. Essa integração requer cuidados especiais para evitar conflitos na interpolação de variáveis e estruturas de controle.
- **Validação em Múltiplos Níveis:** Foi necessário implementar validação de dados tanto no *front-end* (para resposta imediata ao usuário) quanto no *back-end* (para garantir segurança e integridade). Garantir consistência entre essas duas camadas foi essencial.
- **Design Responsivo:** Assegurar que a interface funcionasse bem em diferentes tamanhos de ecrã — desde dispositivos móveis até computadores de mesa — exigiu uma abordagem cuidadosa no uso de classes de estilo e testes visuais constantes.

- **Gerenciamento de Estado da Aplicação:** Manter o estado sincronizado entre diferentes componentes React e as páginas renderizadas pelo Flask exigiu uma arquitetura clara, especialmente para manter os dados do usuário e os resultados das operações consistentes.
- **Tratamento de Erros e Feedback ao Usuário:** Garantir mensagens de erro claras, úteis e exibidas no momento certo foi essencial para proporcionar uma boa experiência ao usuário, principalmente em cenários de entrada inválida ou ausência de dados.

8.3 Soluções Arquiteturais

Para superar os desafios apresentados, foram adotadas soluções específicas que tornaram o desenvolvimento mais eficiente e a aplicação mais robusta:

- **Arquitetura Híbrida Flask + React:** Incorporamos componentes React dentro das páginas geradas com Jinja2, utilizando *scripts* dinâmicos e pontos de montagem (*mount points*) para integrar funcionalidades mais interativas ao *front-end* sem abrir mão da simplicidade do Flask.
- **Uso do Tailwind CSS:** A escolha do Tailwind CSS como framework de estilos permitiu uma construção rápida, coerente e responsiva da interface, com alto grau de customização e sem necessidade de escrever CSS manual repetitivo.
- **Validação Robusta e Coerente:** Foram criadas funções de validação tanto no cliente quanto no servidor, com mensagens harmonizadas e lógica centralizada, o que reduziu a duplicidade de código e facilitou a manutenção.
- **Navegação Fluida e Intuitiva:** A estrutura da aplicação foi organizada de forma que a navegação entre páginas e funcionalidades fosse clara para o usuário, com rotas bem definidas, componentes reutilizáveis e layouts consistentes.

9 Conclusão

O projeto demonstrou a capacidade de desenvolver uma aplicação web moderna e funcional, integrando tecnologias de *back-end* e *front-end* de forma eficiente. A combinação de Flask como servidor *back-end* e React como biblioteca *front-end* resultou numa solução robusta e escalável para processamento e apresentação de operações com sistemas numéricos.

A implementação das funcionalidades de Código de *Gray*, conversões numéricas e operações aritméticas através de uma interface web responsiva destaca a versatilidade da arquitetura escolhida. O projeto contribuiu significativamente para a compreensão de desenvolvimento web full-stack, fortalecendo conhecimentos em integração de tecnologias, design de APIs web, e criação de interfaces utilizador modernas.

A experiência adquirida no desenvolvimento desta aplicação web proporcionou uma compreensão prática dos desafios e soluções no desenvolvimento de ecossistemas web modernos, habilitando-a para projetos de maior complexidade no futuro.

10 Referências Bibliográficas

- Flask Documentation. The Pallets Projects.
  Disponível em: `https://flask.palletsprojects.com/`
- React Documentation. Meta Open Source.
  Disponível em: `https://react.dev/`
- Tailwind CSS Documentation. Tailwind Labs.
  Disponível em: `https://tailwindcss.com/`
- Python Programming for Beginners. Robert Campbell, AMZ Publishing, 2021.
- Learning React: Modern Patterns for Developing React Apps. Alex Banks, Eve Porcello. O'Reilly Media, 2020.
- Head First Python, 2nd Edition. Paul Barry. O'Reilly Media, Inc. 2016. ISBN: 9781491919538.

Acrónimos

API Application Programming Interface

CSS Cascading Style Sheets

DOM Document Object Model

HTML HyperText Markup Language

UBI Universidade da Beira Interior

11 Anexos

11.1 Estrutura de Ficheiros do Projeto

```
sourcecode/
├── app.py ..... Aplicação Flask principal
├── templates/ ..... Arquivos HTML
│   ├── index.html ..... Página inicial
│   ├── gray.html ..... Página de geração de tabelas Gray
│   ├── conversao.html ..... Página de conversão entre sistemas
│   └── operacoes.html ..... Página de operações aritméticas
└── static/ ..... Arquivos estáticos
    └── images/
        ├── logo_ubi_branca.png ..... Logo da UBI
        └── fundo_ubi.png ..... Fundo Imagem UBI
```

11.2 Interfaces da Aplicação

A aplicação web apresenta interfaces modernas e responsivas, com:

- **Página Inicial:** Menu principal com navegação para as três funcionalidades principais
- **Interface Tabela de *Gray*:** Formulário para número de bits e apresentação da tabela gerada
- **Interface Conversões:** Seleção de bases origem/destino e campo para valor a converter
- **Interface Operações:** Seleção de sistema numérico, operação e campos para operandos

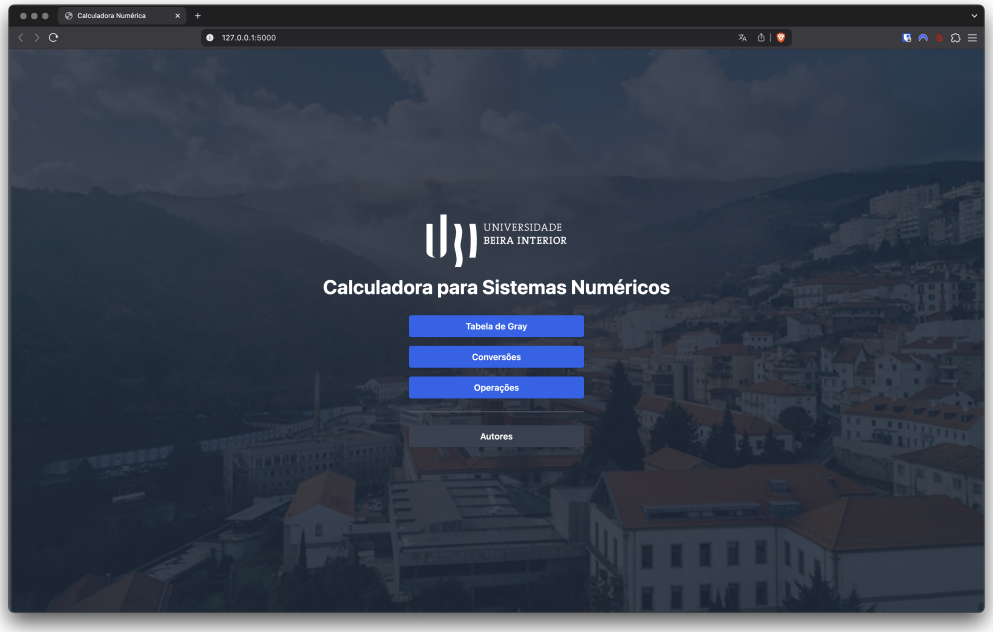


Figura 1: Menu principal com que o utilizador se depara

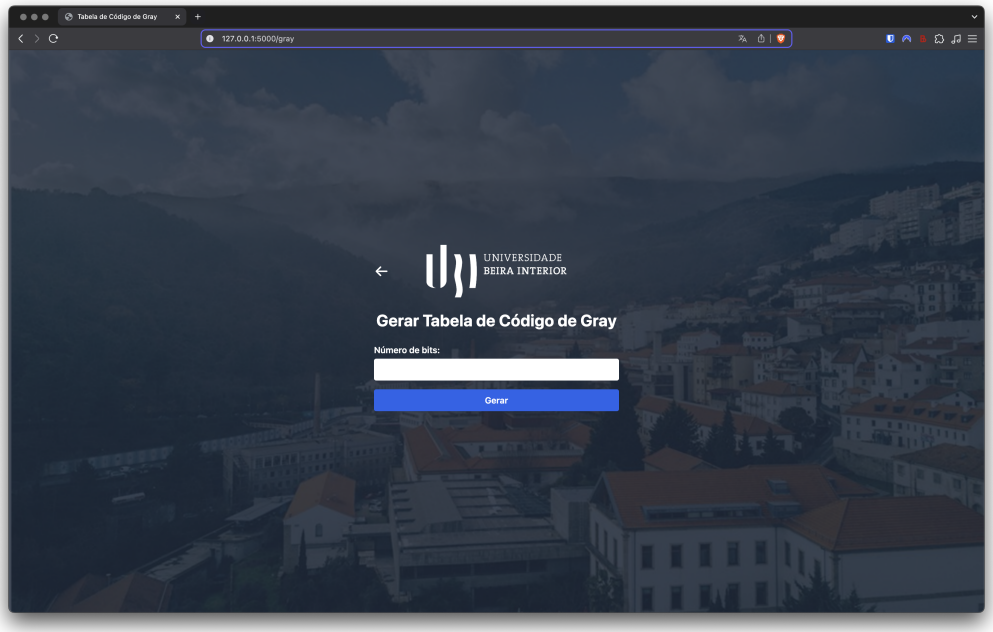


Figura 2: Menu para a criação de tabelas de *Gray*

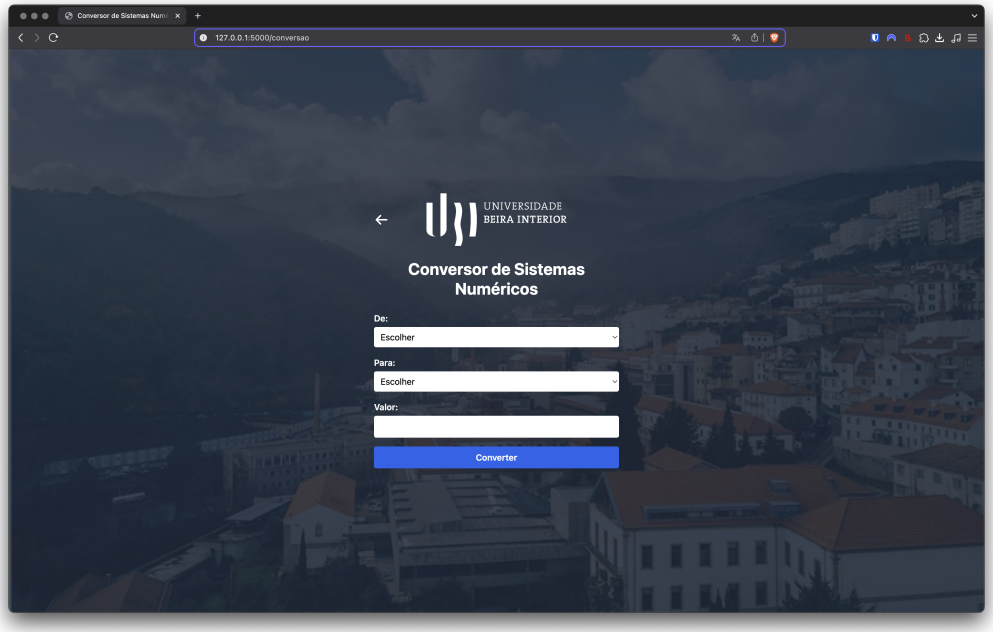


Figura 3: Página de conversões entre sistemas numéricos

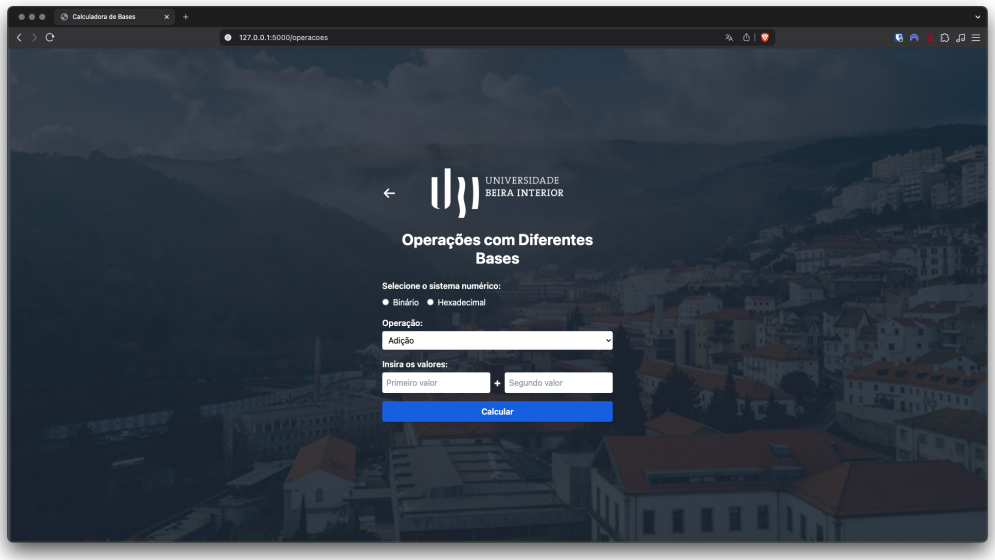


Figura 4: Página de operações aritméticas

Todas as interfaces incorporam:

- Design responsivo adaptável a dispositivos móveis e desktop;
- Feedback visual para ações do utilizador;
- Validação de entrada em tempo real;
- Navegação intuitiva entre funcionalidades.